EDA Notebook

---

0. Setup Environment

In [ ]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.9/12.9 MB 88.4 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.9/4.9 MB 94.3 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are i
nstalled. This behaviour is the source of the following dependency conflicts.
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatib
le.
hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
Mounted at /content/gdrive

You can now save your data files in: /content/gdrive/MyDrive/36106/assignment/AT3/data
```

Student Information

In [ ]:
```
# <Student to fill this section>
group_name = "36106-26AU-AT3-Group 24"
student_name = "Nana Ama Goldwater"
student_id = "26137455"
```

In [ ]:
```
# Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

group_name

36106-26AU-AT3-Group 24

In [ ]:
```
# Do not modify this code
print_tile(size="h1", key='student_name', value=student_name)
```

student_name

Nana Ama Goldwater

In [ ]:
```
# Do not modify this code
print_tile(size="h1", key='student_id', value=student_id)
```

student_id

26137455

---

0. Python Packages

0.a Install Additional Packages

> If you are using additional packages, you need to install them here using the command: `! pip install <package_name>`

In [ ]:
```
# <Student to fill this section>
# No additional packages required. pandas and altair (pre-installed in Colab) are sufficient
# for the descriptive statistics and visualisations performed in this notebook.
```

0.b Import Packages

In [ ]:
```
# <Student to fill this section>
import re
import pandas as pd
import altair as alt

# Show all columns when displaying DataFrames
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 200)
```

B. Data Understanding

In [ ]:
```
# Do not modify this code
try:
  df = pd.read_csv(at.folder_path / "unit_measure.csv")
except Exception as e:
  print(e)
```

```
[Errno 2] No such file or directory: '/content/gdrive/MyDrive/36106/assignment/AT3/data/unit_meas
ure.csv'
```

B.1 Explore Dataset

In [ ]:
```
# <Student to fill this section>
# Inspect the structure: shape, column types, non-null counts.
df = pd.read_csv("/content/unit_measure.csv")
print(f"Dataset shape: {df.shape[0]} rows x {df.shape[1]} columns\n")
df.info()
```

```
Dataset shape: 38 rows x 2 columns

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 38 entries, 0 to 37
Data columns (total 2 columns):
 #   Column             Non-Null Count  Dtype 
---  ------             --------------  ----- 
 0   unit_measure_code  38 non-null     object
 1   name               38 non-null     object
dtypes: object(2)
memory usage: 740.0+ bytes
```

In [ ]:
```
# Preview the first and last rows of the dataset.
display(df.head())
display(df.tail())

# Missing values, exact-duplicate rows and per-column uniqueness.
print("\nMissing values per column:")
print(df.isnull().sum())

print(f"\nExact duplicate rows: {df.duplicated().sum()}")
print(f"Unique unit_measure_code values: {df['unit_measure_code'].nunique()} / {len(df)}")
print(f"Unique name values:              {df['name'].nunique()} / {len(df)}")

# Categorical-style summary (object columns).
display(df.describe(include='all'))
```

	unit_measure_code	name
0	ee0468c7-6417-489a-b233-da57ab966174	Boxes
1	7768b253-7430-402f-b90f-1e025c57459b	Bottle
2	1f485b4e-fc4d-47b2-ab57-af19b21eee40	Celsius
3	e87c6cb9-24ca-405a-996c-8f59ab7730fa	Canister
4	91fa7aa6-72aa-40a6-822f-8b80aa9f6376	Carton

	unit_measure_code	name
33	6ad6e4b7-e1c2-42f2-90f6-306f08b41623	Pack
34	2eb286e8-8050-4cf5-a260-fd2cedb6e1f4	Pallet
35	5885bdb8-189e-4001-9f96-9cffd780b130	Piece
36	14266c5d-bb1d-4b58-9d8d-f13f6eb35863	Percentage
37	a24c076e-f88e-44e9-811b-b4a3da40d463	Pint, US liquid

```
Missing values per column:
unit_measure_code    0
name                 0
dtype: int64

Exact duplicate rows: 0
Unique unit_measure_code values: 38 / 38
Unique name values:              38 / 38
```

	unit_measure_code	name
count	38	38
unique	38	38
top	ee0468c7-6417-489a-b233-da57ab966174	Boxes
freq	1	1

In [ ]:
```
# <Student to fill this section>
dataset_insights = """
The dataset `unit_measure.csv` is a small reference / lookup table with 38 rows and 2 columns:
  - `unit_measure_code`: object; a unique identifier formatted as a UUID (36 characters).
  - `name`: object; the human-readable label of the unit of measure (e.g. 'Kilogram', 'Liter').

Quality / completeness:
  - No missing values in either column.
  - No exact duplicate rows.
  - All 38 `unit_measure_code` values are unique, and all 38 `name` values are unique on exact ma
  - The table behaves like a dimension / lookup table rather than a transactional dataset.

Issues / things to watch:
  - The 38 rows are very few. This table is intended to be joined onto a larger fact table
    (e.g. inventory, sales, products) via `unit_measure_code`. EDA in isolation is limited.
  - The `name` column mixes units from very different physical dimensions: weight (Kilogram,
    Gram), length (Meter, Inch), area (Square meter), volume (Liter, Cubic meter), counts
    (Each, Dozen, Pack), temperature (Celsius) and even Percentage. Downstream code that
    converts between units must therefore validate that the unit is dimensionally compatible.
  - Near duplicate entries exist (see B.4): 'Cubic meter' and 'Cubic meters' refer to the
    same physical unit but are stored under two different UUIDs.
  - One value contains a comma and is therefore CSV-quoted ('Pint, US liquid'); this is
    handled correctly by pandas but is worth flagging for any non-pandas consumer.
"""
```

In [ ]:
```
# Do not modify this code
print_tile(size="h3", key='dataset_insights', value=dataset_insights)
```

The dataset `unit_measure.csv` is a small reference / lookup table with 38 rows and 2 columns: - `unit_measure_code`: object; a unique identifier formatted as a UUID (36 characters). - `name`: object; the human-readable label of the unit of measure (e.g. 'Kilogram', 'Liter'). Quality / completeness: - No missing values in either column. - No exact duplicate rows. - All 38 `unit_measure_code` values are unique, and all 38 `name` values are unique on exact match. - The table behaves like a dimension / lookup table rather than a transactional dataset. Issues / things to watch: - The 38 rows are very few. This table is intended to be joined onto a larger fact table (e.g. inventory, sales, products) via `unit_measure_code`. EDA in isolation is limited. - The `name` column mixes units from very different physical dimensions: weight (Kilogram, Gram), length (Meter, Inch), area (Square meter), volume (Liter, Cubic meter), counts (Each, Dozen, Pack), temperature (Celsius) and even Percentage. Downstream code that converts between units must therefore validate that the unit is dimensionally compatible. - Near duplicate entries exist (see B.4): 'Cubic meter' and 'Cubic meters' refer to the same physical unit but are stored under two different UUIDs. - One value contains a comma and is therefore CSV-quoted ('Pint, US liquid'); this is handled correctly by pandas but is worth flagging for any non-pandas consumer.

B.2 Explore Feature of Interest `unit_measure_code`

In [ ]:
```
# <Student to fill this section>
# Basic profile of the identifier column.
print(f"Total values        : {len(df)}")
print(f"Unique values       : {df['unit_measure_code'].nunique()}")
print(f"Missing values      : {df['unit_measure_code'].isnull().sum()}")
print(f"Min length          : {df['unit_measure_code'].str.len().min()}")
print(f"Max length          : {df['unit_measure_code'].str.len().max()}")

# Check that every value matches the canonical UUID pattern.
uuid_re = re.compile(r'^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$')
df['is_valid_uuid'] = df['unit_measure_code'].apply(lambda x: bool(uuid_re.match(str(x))))
print(f"\nProportion matching UUID v-format: {df['is_valid_uuid'].mean():.0%}")

# Show a few samples.
display(df[['unit_measure_code']].head())
```

```
Total values        : 38
Unique values       : 38
Missing values      : 0
Min length          : 36
Max length          : 36

Proportion matching UUID v-format: 100%
```

	unit_measure_code
0	ee0468c7-6417-489a-b233-da57ab966174
1	7768b253-7430-402f-b90f-1e025c57459b
2	1f485b4e-fc4d-47b2-ab57-af19b21eee40
3	e87c6cb9-24ca-405a-996c-8f59ab7730fa
4	91fa7aa6-72aa-40a6-822f-8b80aa9f6376

In [ ]:
```
# Visualise the distribution of string lengths for `unit_measure_code`.
# A clean UUID column should produce a single bar at length = 36.
len_df = (
    df.assign(code_length=df['unit_measure_code'].str.len())
      .groupby('code_length').size().reset_index(name='count')
)

alt.Chart(len_df).mark_bar().encode(
    x=alt.X('code_length:O', title='Length of unit_measure_code'),
    y=alt.Y('count:Q',       title='Number of records'),
```

```
    tooltip=['code_length', 'count']
).properties(
    width=400, height=250,
    title='Distribution of unit_measure_code string lengths'
)
```

Out[ ]:

In [ ]:
```
# <Student to fill this section>
feature_1_insights = """
`unit_measure_code` is the primary key of this lookup table.

Distribution / format:
  - 38 values, all unique, no missing data.
  - Every value is exactly 36 characters long and matches the canonical UUID v-format
    (8-4-4-4-12 hex pattern with hyphens), so the column is well-formed.
  - Because each value is unique and has no inherent ordering or magnitude, the column has
    no statistical distribution to analyse — only its format and uniqueness matter.

Limitations / issues:
  - As a UUID, the code carries no business meaning it cannot be parsed for unit type or
    dimension. All semantic information lives in `name`.
  - The column is unsuitable as a model feature (high-cardinality unique identifier).
    Its role is strictly to act as a join key onto fact tables; for any modelling task it
    should be dropped or replaced by an attribute derived from `name`.
  - If this dataset is ever appended to, uniqueness must be re-checked: any duplicate code
    here would silently corrupt joins downstream.
"""
```

In [ ]:
```
# Do not modify this code
print_tile(size="h3", key='feature_1_insights', value=feature_1_insights)
```

feature_1_insights

`unit_measure_code` is the primary key of this lookup table. Distribution / format: - 38 values, all unique, no missing data. - Every value is exactly 36 characters long and matches the canonical UUID v-format (8-4-4-4-12 hex pattern with hyphens), so the column is well-formed. - Because each value is unique and has no inherent ordering or magnitude, the column has no statistical distribution to analyse — only its format and uniqueness matter. Limitations / issues: - As a UUID, the code carries no business meaning it cannot be parsed for unit type or dimension. All semantic information lives in `name`. - The column is unsuitable as a model feature (high-cardinality unique identifier). Its role is strictly to act as a join key onto fact tables; for any modelling task it should be dropped or replaced by an attribute derived from `name`. - If this dataset is ever appended to, uniqueness must be re-checked: any duplicate code here would silently corrupt joins downstream.

B.3 Explore Feature of Interest `name`

In [ ]:
```
# <Student to fill this section>
# Profile of the human-readable name column.
print(f"Total values   : {len(df)}")
print(f"Unique values  : {df['name'].nunique()}")
print(f"Missing values : {df['name'].isnull().sum()}")
print(f"\nName length statistics:")
print(df['name'].str.len().describe().round(2))

# Sorted list of all names for visual inspection.
print("\nAll unit names (sorted):")
for n in sorted(df['name'].tolist()):
    print(" -", n)
```

```
Total values   : 38
Unique values  : 38
Missing values : 0

Name length statistics:
count    38.00
mean      8.47
std       4.03
min       4.00
25%       5.00
50%       8.00
75%      10.00
max      20.00
Name: name, dtype: float64

All unit names (sorted):
 - Bottle
 - Boxes
 - Canister
 - Carton
 - Case
 - Celsius
 - Centimeter
 - Container
 - Crate
 - Cubic centimeter
 - Cubic decimeter
 - Cubic foot
 - Cubic meter
 - Cubic meters
 - Decimeter
 - Dozen
 - Each
 - Gallon
 - Gram
 - Inch
 - Kilogram
 - Kilogram/cubic meter
 - Kilometer
 - Kiloton
 - Liter
 - Meter
 - Milligram
 - Milliliter
 - Millimeter
 - Ounces
 - Pack
 - Pallet
 - Percentage
 - Piece
 - Pint, US liquid
 - Square centimeter
 - Square meter
 - US pound
```

In [ ]:
```python
# Visualise the distribution of name lengths to spot outliers
# (very short like 'Each' vs very long like 'Kilogram/cubic meter').
name_len_df = df.assign(name_length=df['name'].str.len())

hist = alt.Chart(name_len_df).mark_bar().encode(
    x=alt.X('name_length:Q', bin=alt.Bin(step=2), title='Name length (characters)'),
    y=alt.Y('count():Q',                          title='Number of names'),
    tooltip=[alt.Tooltip('count():Q', title='count')]
).properties(
    width=450, height=250,
    title='Distribution of `name` string lengths'
)
hist
```

Out[ ]:

In [ ]:
```python
# <Student to fill this section>
feature_n_insights_unused = ""  # placeholder, real insights below
feature_2_insights = """
`name` is the descriptive label of the unit of measure.
```

```
Distribution / values:
  - 38 unique string values, no missing data.
  - Lengths range from 4 characters ('Each', 'Pack', 'Gram') to 20 characters
    ('Kilogram/cubic meter'), with a mean of ~8.5 characters.
  - The values cover several physical dimensions:
      * Weight        : Gram, Kilogram, Kiloton, Milligram, Ounces, US pound
      * Length        : Centimeter, Decimeter, Inch, Kilometer, Meter, Millimeter
      * Area          : Square centimeter, Square meter
      * Volume        : Cubic centimeter, Cubic decimeter, Cubic foot, Cubic meter,
                        Cubic meters, Gallon, Liter, Milliliter, Pint US liquid
      * Container     : Bottle, Boxes, Canister, Carton, Case, Container, Crate, Pack, Pallet
      * Count         : Dozen, Each, Piece
      * Density       : Kilogram/cubic meter
      * Temperature   : Celsius
      * Ratio         : Percentage

Limitations / issues:
  - The column is free-text with inconsistent style: some entries are plural ('Boxes',
    'Ounces', 'Cubic meters') while equivalent ones are singular ('Carton', 'Cubic meter').
  - Mixing fundamentally different physical dimensions in a single lookup means downstream
    code MUST check dimensional compatibility before any unit conversion. Adding 1 Celsius
    to 1 Kilogram is meaningless but the schema does not prevent it.
  - 'Container' units (Box, Carton, Crate, Pallet, ...) have no fixed physical size, which
    makes them non-convertible to SI units without extra context (item, supplier, ...).
  - 'Each', 'Piece' and 'Dozen' are counts rather than measures and may belong in a
    separate dimension.
  - There is no `category` / `dimension` column to disambiguate — that has to be derived
    manually (see B.4).
"""
```

In [ ]:
```
# Do not modify this code
print_tile(size="h3", key='feature_2_insights', value=feature_2_insights)
```

feature_2_insights

`name` is the descriptive label of the unit of measure. Distribution / values: - 38 unique string values, no missing data. - Lengths range from 4 characters ('Each', 'Pack', 'Gram') to 20 characters ('Kilogram/cubic meter'), with a mean of ~8.5 characters. - The values cover several physical dimensions: * Weight : Gram, Kilogram, Kiloton, Milligram, Ounces, US pound * Length : Centimeter, Decimeter, Inch, Kilometer, Meter, Millimeter * Area : Square centimeter, Square meter * Volume : Cubic centimeter, Cubic decimeter, Cubic foot, Cubic meter, Cubic meters, Gallon, Liter, Milliliter, Pint US liquid * Container : Bottle, Boxes, Canister, Carton, Case, Container, Crate, Pack, Pallet * Count : Dozen, Each, Piece * Density : Kilogram/cubic meter * Temperature : Celsius * Ratio : Percentage Limitations / issues: - The column is free-text with inconsistent style: some entries are plural ('Boxes', 'Ounces', 'Cubic meters') while equivalent ones are singular ('Carton', 'Cubic meter'). - Mixing fundamentally different physical dimensions in a single lookup means downstream code MUST check dimensional compatibility before any unit conversion. Adding 1 Celsius to 1 Kilogram is meaningless but the schema does not prevent it. - 'Container' units (Box, Carton, Crate, Pallet, ...) have no fixed physical size, which makes them non-convertible to SI units without extra context (item, supplier, ...). - 'Each', 'Piece' and 'Dozen' are counts rather than measures and may belong in a separate dimension. - There is no `category` / `dimension` column to disambiguate — that has to be derived manually (see B.4).

B.4 Explore Feature of Interest `name` — Data Quality & Derived Category

In [ ]:
```
# <Student to fill this section>
# 1) Hunt for near-duplicate names that differ only by case or trailing 's'.
norm = (
    df['name']
      .str.lower()
      .str.strip()
      .str.replace(r's$', '', regex=True)   # strip trailing plural 's'
)
df_norm = df.assign(name_norm=norm)
```

```
near_dups = df_norm[df_norm.duplicated('name_norm', keep=False)].sort_values('name_norm')

print("Near-duplicate names (different UUIDs, same normalised name):")
display(near_dups[['unit_measure_code', 'name', 'name_norm']])
```

```
Near-duplicate names (different UUIDs, same normalised name):
```

	unit_measure_code	name	name_norm
5	1f5f8ee7-58c4-4c4e-b8db-b7c1025b3400	Cubic meters	cubic meter
28	8cb5f9fe-f273-42f3-96e4-1f93e26bb318	Cubic meter	cubic meter

In [ ]:

```
# 2) Derive a `category` column by keyword matching, then visualise the breakdown.
def categorise(name: str) -> str:
    n = name.lower()
    if 'cubic' in n or n in {'liter', 'milliliter', 'gallon', 'pint, us liquid'}:
        return 'Volume'
    if 'square' in n:
        return 'Area'
    if n in {'gram', 'kilogram', 'milligram', 'kiloton', 'ounces', 'us pound'}:
        return 'Weight'
    if n in {'centimeter', 'decimeter', 'inch', 'kilometer', 'meter', 'millimeter'}:
        return 'Length'
    if n in {'kilogram/cubic meter'}:
        return 'Density'
    if n in {'celsius'}:
        return 'Temperature'
    if n in {'percentage'}:
        return 'Ratio'
    if n in {'each', 'piece', 'dozen'}:
        return 'Count'
    return 'Container/Packaging'

df_cat = df.assign(category=df['name'].apply(categorise))
cat_counts = (
    df_cat.groupby('category').size()
          .reset_index(name='count')
          .sort_values('count', ascending=False)
)
display(cat_counts)

alt.Chart(cat_counts).mark_bar().encode(
    x=alt.X('count:Q',          title='Number of units'),
    y=alt.Y('category:N',       sort='-x', title='Derived category'),
    tooltip=['category', 'count']
).properties(
    width=450, height=300,
    title='Units per derived measurement category'
)
```

	category	count
6	Volume	10
1	Container/Packaging	9
7	Weight	6
3	Length	6
2	Count	3
0	Area	2
5	Temperature	1
4	Ratio	1

Out[ ]:

In [ ]:

```
# <Student to fill this section>
feature_n_insights = """
Data quality and a derived `category` view of the `name` column.

Near-duplicate detection:
  - After normalising names (lowercase + strip trailing 's') one collision is found:
```

```
      * 'Cubic meter'  -> 8cb5f9fe-f273-42f3-96e4-1f93e26bb318
      * 'Cubic meters' -> 1f5f8ee7-58c4-4c4e-b8db-b7c1025b3400
    These are the same physical unit recorded twice under two different UUIDs. This is a
    real data-quality issue: any join from a fact table will be split between the two
    codes, fragmenting reporting. Recommendation: deprecate one UUID and remap fact-table
    references to the survivor, then enforce a uniqueness constraint on a normalised name.
  - 'Boxes' and 'Ounces' are plural-only forms with no singular twin in the table; they
    are not duplicates but the inconsistent singular/plural styling is worth standardising.

Derived category column:
  - The 38 units fall into 8 dimensional groups. The largest group is
    Container/Packaging (Bottle, Boxes, Canister, Carton, Case, Container, Crate, Pack,
    Pallet) followed by Volume (9), Length (6) and Weight (6). Density, Temperature,
    Ratio and Count are each represented by only a few rows.
  - Container/Packaging units have no fixed SI size and so cannot be converted
    arithmetically to Volume or Weight without product-specific context.
  - Temperature ('Celsius') and Ratio ('Percentage') being present in a 'unit_measure'
    table is unusual. These may have been added for convenience but they do not
    behave like other units (e.g. they are not additive in the same way), which is a
    schema-design concern more than a row-level error.

Limitations:
  - The category mapping above is rule-based and hand-curated; any new unit added to
    the dataset would need a corresponding rule. A dedicated `category` column in the
    source data would remove this fragility.
"""
```

In [ ]:
```
# Do not modify this code
print_tile(size="h3", key='feature_n_insights', value=feature_n_insights)
```

feature_n_insights

Data quality and a derived `category` view of the `name` column. Near-duplicate detection: - After normalising names (lowercase + strip trailing 's') one collision is found: * 'Cubic meter' - 8cb5f9fe-f273-42f3-96e4-1f93e26bb318 * 'Cubic meters' - 1f5f8ee7-58c4-4c4e-b8db-b7c1025b3400 These are the same physical unit recorded twice under two different UUIDs. This is a real data-quality issue: any join from a fact table will be split between the two codes, fragmenting reporting. Recommendation: deprecate one UUID and remap fact-table references to the survivor, then enforce a uniqueness constraint on a normalised name. - 'Boxes' and 'Ounces' are plural-only forms with no singular twin in the table; they are not duplicates but the inconsistent singular/plural styling is worth standardising. Derived category column: - The 38 units fall into 8 dimensional groups. The largest group is Container/Packaging (Bottle, Boxes, Canister, Carton, Case, Container, Crate, Pack, Pallet) followed by Volume (9), Length (6) and Weight (6). Density, Temperature, Ratio and Count are each represented by only a few rows. - Container/Packaging units have no fixed SI size and so cannot be converted arithmetically to Volume or Weight without product-specific context. - Temperature ('Celsius') and Ratio ('Percentage') being present in a 'unit_measure' table is unusual. These may have been added for convenience but they do not behave like other units (e.g. they are not additive in the same way), which is a schema-design concern more than a row-level error. Limitations: - The category mapping above is rule-based and hand-curated; any new unit added to the dataset would need a corresponding rule. A dedicated `category` column in the source data would remove this fragility.

In [3]:
```
!pip install -q utstd
from utstd.ipyrenders import print_tile

ethics_lookup_insights = """
Ethics — product-side reference data

This table contains no personal or community-related information. It is a
lookup of physical units of measure. No ethics or privacy concerns
attach to this table in isolation.

Worth noting only:
  - The near-duplicate 'Cubic meter' / 'Cubic meters' issue is a data-
    quality concern (joins fragment across two UUIDs), not an ethics
```

```
    issue. It is documented in B.4 as a recommendation to deprecate one
    UUID before any downstream reporting.
"""
print_tile(size="h3", key='ethics_lookup_insights', value=ethics_lookup_insights)
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/12.9 MB ? eta -:--:--
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.9/12.9 MB 238.3 MB/s eta 0:00:01
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.9/12.9 MB 254.9 MB/s eta 0:00:01
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.9/12.9 MB 118.1 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/4.9 MB ? eta -:--:--
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.9/4.9 MB 296.2 MB/s eta 0:00:01
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.9/4.9 MB 130.9 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are i
nstalled. This behaviour is the source of the following dependency conflicts.
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatib
le.
hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
```

ethics_lookup_insights

Ethics — product-side reference data This table contains no personal or community-related information. It is a lookup of physical units of measure. No ethics or privacy concerns attach to this table in isolation. Worth noting only: - The near-duplicate 'Cubic meter' / 'Cubic meters' issue is a data-quality concern (joins fragment across two UUIDs), not an ethics issue. It is documented in B.4 as a recommendation to deprecate one UUID before any downstream reporting.